Zenith: programs, experiments and useful memory

Will Cook • 19 October 2026

Abstract. Zenith asks whether the experience an AI gains while testing programs can make it test the next, unseen program better. We fix one model, DeepSeek V4.1 Flash, and give it a hard budget: five inputs to expose a hidden fault, with only the function's specification in view. The faults are real: 535 behaviour classes of solutions written by three code models that pass every original HumanEval test yet fail an EvalPlus input. On 28 practice problems the model's misses, together with the inputs that would have exposed them, form a record; the model then distils the record into a short playbook. Shown that record, the model exposed 77.0% of hidden fault classes on 58 held-out problems, against 69.2% when shown nothing: a gain of 7.9 points (95% interval 1.4 to 14.7; better on 20 problems, worse on 6). About half of the gain comes from very large inputs that make slow faulty programs miss a one-second limit; counting only wrong answers and exceptions, the gain is 3.5 points (−2.1 to 8.9). Playbooks the model wrote from the same record, generic advice and imagining faults before testing gave no measurable gain, and with reasoning switched off no form of experience helped. A development pilot had suggested a 16-point gain for the first playbook; the held-out run did not reproduce it. Model weights never change; only what the tester is shown does. The result holds for one benchmark of short Python functions and one model, under a protocol committed before the held-out run.

1. Introduction. When a tester finds a mistake in a program, it learns something about how programs go wrong. Most automated testing throws that away: each program is tested afresh. We ask a narrow, measurable version of a broader question. If a model keeps the record of its earlier testing, including the mistakes it failed to expose, does it choose better tests for programs it has never seen?

Why a small budget. With unlimited inputs, random or exhaustive testing eventually exposes most faults, and the question disappears. We give the tester five inputs per program. Choosing five inputs that expose a mistake nobody has described is the part of testing that needs judgement: where are the boundaries, which rules interact, what would a careless implementation get wrong?

Why model-written faults. Hand-made faults risk measuring what the study author expected. Mechanical mutants are standard but are an imperfect proxy for real faults [1, §3]. We instead use mistakes that code models actually made: EvalPlus showed that many solutions accepted by HumanEval's original tests are wrong [2, §1; §3]. Each such solution is a fault that a competent-looking test suite missed.

Contributions.

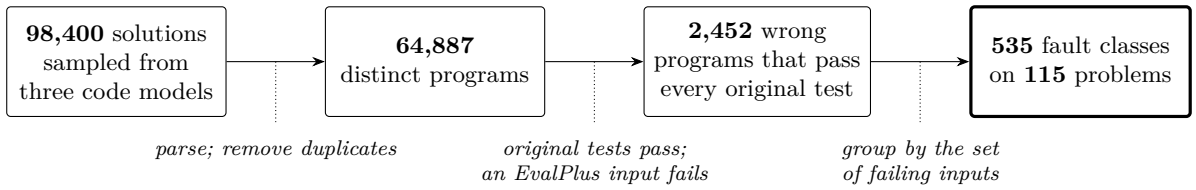
- **A population of natural faults.** We execute 64,887 distinct sampled solutions and keep the 2,452 wrong programs that every original test accepts: 535 behaviour classes on 115 problems, selected by a fixed rule.
- **A controlled test of kept experience.** Seven conditions share one model, prompt frame and five-input budget, and differ only in what the tester is shown: nothing, generic advice, its practice record, a playbook it wrote, a revised playbook, or a method that imagines faults before choosing inputs.
- **Results.** On 58 held-out problems the raw practice record raises the share of faults exposed from 69.2% to 77.0%; written playbooks, advice and imagined faults do not measurably help, and nothing helps with reasoning switched off. The model's own five inputs beat five random EvalPlus inputs by 26 points.

The study is part of Zenith, a research testbed about retaining programs, experiments and the questions that expose their limits. Section 5 summarises the earlier Zenith studies, including the negative results this study was designed to move beyond.

2. Programs, faults and the testing task. Programs. We use the 164 HumanEval problems

as extended by EvalPlus [2, §2]. Each problem supplies a public specification (signature, docstring and examples), a reference solution, the original HumanEval test inputs and about a thousand augmented inputs that EvalPlus generated by type-aware mutation of seed inputs. EvalPlus also supplies input contracts: assertions that a legal input must satisfy.

Natural faults. EvalPlus released code sampled from many models. We take the 98,400 solutions sampled at temperature 0.8 from ChatGPT, CodeGen-16B and InCoder-6B. A solution is a *natural fault* when it returns the reference's answer on every original HumanEval input but differs on at least one augmented input. These are mistakes that real code models made and that the original human-written tests did not catch. Faulty solutions that fail exactly the same augmented inputs form one *fault class*. The rule is fixed and mechanical; neither we nor the tested model choose faults.



Fault population. Executing every distinct sampled program on the original tests and on EvalPlus's augmented inputs leaves 2,452 wrong programs that the original tests accept, in 535 behaviour classes on 115 of the 164 problems. Eight batches of 40 programs exceeded the run's memory or time bounds and are excluded and recorded.

As a secondary population we apply single-site mutation operators (relational, arithmetic, logical, constant and condition changes) to each reference and keep mutants with the same property. Mutant detection correlates with real-fault detection in Java projects, but about a quarter of real faults are not coupled to common mutants [1, §3]; we therefore treat model-written faults as primary.

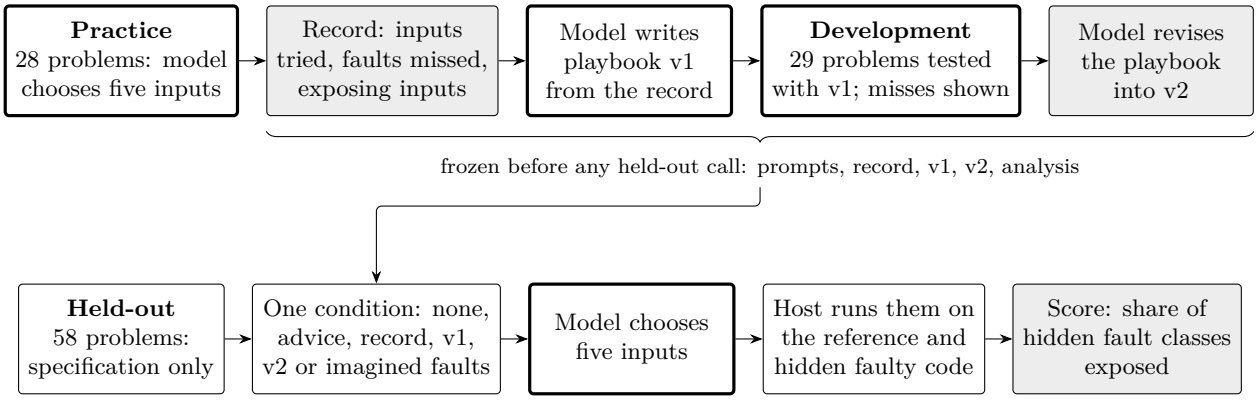
The testing task. A tester sees one problem's specification and its input contract, and chooses $K = 5$ inputs. The host executes each legal input on the reference and on every hidden fault class of that problem. A fault class is *exposed* when any chosen input yields a different return value, a different exception type, or no answer within one second where the reference answers. Illegal inputs are discarded and still use budget. The tester never sees the faulty code, any output, or the augmented inputs of the problem it is testing. A problem's score is the share of its fault classes exposed; the primary measure is the mean score over held-out problems.

Splits. We order the 115 problems with natural faults by a fixed hash of their identifiers: the first quarter (28) is *practice*, the only source of experience; the next quarter (29) is *development*, used for pilots and for revising the playbook; the remaining half (58) is *held out* and was run once, after the protocol, prompts and analysis were committed.

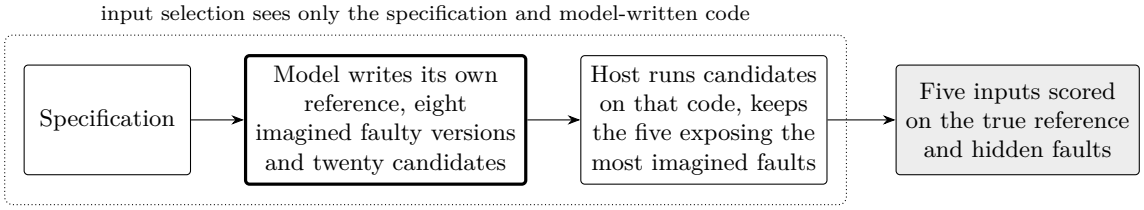
3. What the tester is shown. Every condition uses the same model, prompt frame and budget. Only the material placed before the specification differs.

- **None.** The specification alone.
- **Advice.** A fixed paragraph of generic testing advice: boundaries, ties, ordering, permitted types, size and interacting conditions.
- **Record.** The practice record: for each practice problem, the inputs chosen, how many faults they exposed and up to three missed faults, each with its code and an augmented input that exposes it.
- **Playbook v1.** A playbook of at most 400 words that the model wrote from the practice record, asked to generalise beyond the specific functions.
- **Playbook v2.** The model's revision of v1 after v1 was used on the development problems and its misses were revealed.
- **Imagined faults.** The model writes its own reference, eight plausible faulty versions and twenty candidate inputs; the host keeps the five candidates that expose the most imagined faults (greedy

set cover).



Study design. Experience comes only from practice problems; the revised playbook only from development problems. Prompts, records, playbooks and analysis are committed before any held-out call. Shaded boxes are stored records; heavy boxes are model calls.

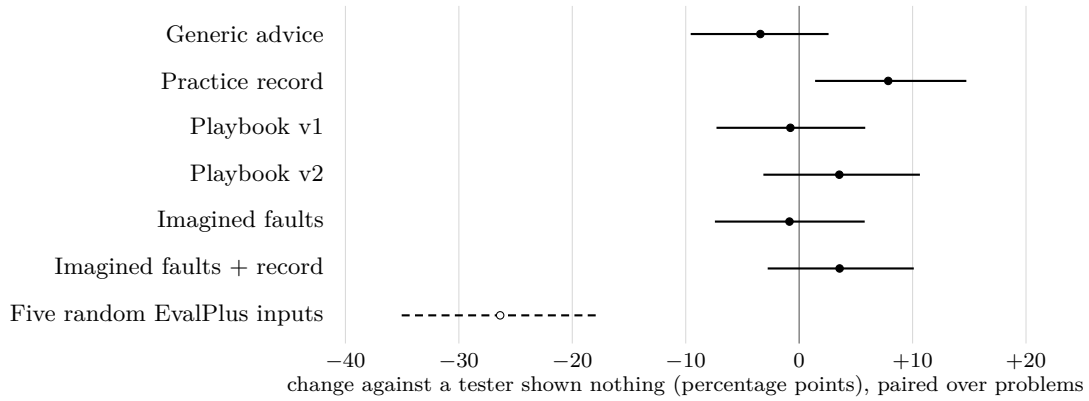


Imagined faults. Selection executes only model-written code and the public input contract. The true reference and hidden faults are used only for scoring, exactly as in the other conditions.

Model and accounting. All conditions use DeepSeek V4.1 Flash (snapshot 1017) through OpenRouter, pinned to one endpoint, with high reasoning effort. A reply whose answer field is empty or lacks the requested block is a delivery failure and is requested again, at most three times, identically in every condition; undelivered attempts remain in the record and the cost total. Every prompt, reply, cost and execution is stored; scoring replays with no network calls. A random baseline draws five augmented inputs uniformly; we report its exact expectation over 200 draws.

4. Results. The table and figure below give the pre-registered comparison. Each condition ran twice on each of the 58 held-out problems; a problem's score is averaged over the two runs, and every interval is a 95% bootstrap over problems, paired against the tester shown nothing. Only the practice record shows a gain whose interval excludes zero. The two playbooks, which the model wrote from that same record, do not. Five inputs drawn at random from EvalPlus's augmented set expose 42.8%, so the five inputs the model chooses unaided are worth about 28 random augmented inputs (69.8% on average), and the five it chooses with its record about 47. Choosing the best five augmented inputs for each problem with hindsight would expose 99.3%, so ample room remains.

Shown to the tester	Exposed (%)	Change vs none	Better / worse problems
None	69.2	–	–
Generic advice	65.8	−3.4 [−9.6, +2.6]	14 / 16
Practice record	77.0	+7.8 [+1.4, +14.7]	20 / 6
Playbook v1	68.4	−0.8 [−7.3, +5.8]	15 / 14
Playbook v2	72.7	+3.5 [−3.1, +10.7]	14 / 11
Imagined faults	68.3	−0.9 [−7.4, +5.8]	14 / 17
Imagined faults + record	72.7	+3.6 [−2.8, +10.1]	17 / 15
Five random EvalPlus inputs	42.8	−26.4 [−35.0, −17.6]	12 / 45



Paired change against a tester shown nothing, 58 held-out problems. Dots are mean differences; bars are 95% bootstrap intervals over problems. The open dot is five random EvalPlus inputs, which uses no model.

An example. HumanEval problem 37 asks for a list whose values at even positions are sorted. One sampled solution begins by asserting that the list is not empty; the specification allows an empty list, so the solution raises an exception where the reference returns an empty list. Shown nothing, the model never tried an empty list in either run. Shown its record, in which practice had revealed missed faults of the same kind, it included the empty list in both runs.

Where the gain comes from. Exposure counts any different return value, different exception, or failure to answer within one second where the reference answers; the frozen scoring code applied this rule, though the protocol's prose named only values and exceptions. The record taught the model to add very large inputs, such as the prime 1,000,000,007 for factorisation, which make slow faulty programs time out. Counting only wrong values and exceptions, the record still leads, 69.8% against 66.3%, but the interval includes zero (−2.1 to 8.9 points; better on 19 problems, worse on 8). Part of the gain is also format: shown nothing, the model occasionally passed arguments in the wrong shape, which the input contract rejects. On the 53 problems where both conditions supplied five valid inputs in both runs, the record's gain is 6.0 points (−0.8 to 13.2).

Reasoning. With reasoning switched off (experiment 2, pre-registered after the main arms were scored), the tester shown nothing exposes 55.9%, 13.3 points below the same prompt with high reasoning. No form of experience helps without reasoning: the record scores 53.5% and the playbooks 49.7% and 48.3%. With reasoning on, the playbooks shorten the model's reasoning (median about 4,400 to 4,900 tokens against 6,400 for nothing), while the record lengthens it (7,300). This is consistent with the record helping because the model reasons over the concrete faults it shows, but the study does not isolate that mechanism.

Development and mechanical faults. On the 29 development problems the first playbook had led by 16.5 points over a single run of the tester shown nothing, whose score there (56.9%) was well below its practice and held-out levels; the held-out run did not reproduce the lead. On mechanical mutants (12 held-out problems) the record exposes 80.8% against 63.0%, with a wide interval (1.2 to 37.5 points).

Cost. The whole study used 1,565 provider calls for $5.70, all on one endpoint: $1.09 for practice, development and the two playbooks, $4.42 for the seven held-out conditions of experiment 1 and $0.19 for experiment 2. The imagined-fault conditions were the most expensive, at about $0.008 per call.

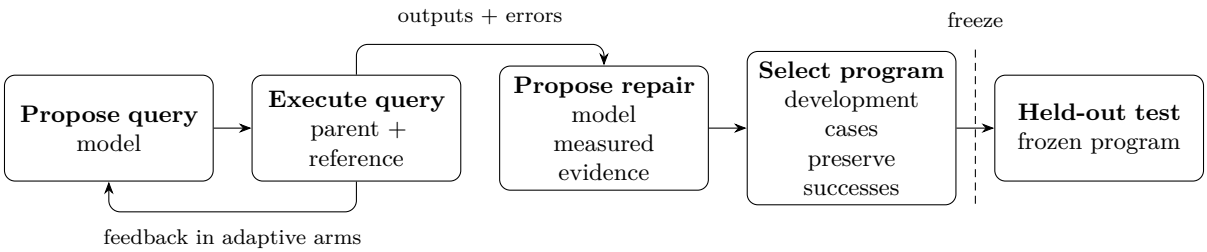
5. Earlier Zenith studies. We represent retained artifacts as (P, E, Q): a program, source-bound observations, and executable tests or distinguishing continuations. A useful addition must improve later behavior or reduce the work required to acquire it. Cached answers, learned representations and better investigation procedures require separate controls. Model weights remain fixed in these studies.

Reusable SQLite testers. Five authored batch clients supply development experience. Two generated testers then run unchanged on five account-transfer clients, four faulty and one correct. Each has six query slots per client; invalid and duplicate queries consume slots. Target execution finishes before a separate 400-case panel opens.

Method	Faults / 4	Native probes	Target model calls
Retained tester	3	5	0
Ordinary code reuse	3	5	0
Same-history fresh	3	15	19
Cold investigation	3	15	15
Fixed tester	4	7	0

Both generated testers repeat one constant query, ignore feedback and spend ten slots on duplicates. DeepSeek V4.1 Flash on Morph uses 46 calls for $0.007117764, including development and compilation. That run asked for low reasoning effort and the provider reported none; the present study uses high effort. The family was inspected during engineering. This establishes executable reuse, not adaptive testing or independent transfer.

Program repair. In the native study, a model receives a task contract, a whole candidate program, examples and the measurements available to its condition. It proposes queries; the host compares candidate and reference answers. A repair request proposes replacement code. Selection preserves every previously correct measured case and the aggregate development result before later panels open.



Native repair: measured evidence informs the proposed repair. Development selection preserves measured successes and requires a non-decreasing aggregate score. Held-out outcomes are withheld from repair.

On twelve parent episodes from SQLite, LRU and UTF-8 contracts, repair without new investigation improves confirmation from 259/576 to 365/576 answers. Fixed testing scores 364, compiled investigation 354, a direct batch 339, feedback-aware two-turn requests 343 and outcome-masked requests 331. There are only three authored mechanisms; case rows are not independent replications. No investigation arm beats repair alone. Ling3.1Flash on DeepInfra uses 130 calls for $0.12982944.

Continuation learning. Predictive state representations encode stochastic state through probabilities of future tests. The cited theorem constructs a representation from a known finite POMDP; it does not supply a black-box learning algorithm [3, Theorem 1; §2]. Our continuation learner instead fits exact integer observations in a rational-linear model. The conceptual connection is the use of future tests to distinguish histories.

Access histories h and continuation tests t index an observation table, $H_{h,t} = f(ht)$, for deterministic observations f. Given exact field-valued answers and a square full-rank table, a counterexample supplies a history–test pair whose nonzero residual increases the rank [4, Proposition 1, pp. 2–3]. Our repair preserves the paired access history and distinguishing test required by this construction. Across eight native configurations both the repaired refinement and a generic bank of one-symbol questions complete every fit; generic questions cost 11,379 acquisition source executions versus 11,395, and later finite checks retain errors. Useful questions have not yet made that architecture economical.

6. Related work. Test adequacy and faults. EvalPlus extends HumanEval with inputs from model-written seeds and type-aware mutation, and shows that pass rates fall when the extra inputs are used [2, §2.1; §3]; we reuse its inputs as the ground truth that defines a fault. Just et al. find mutant detection positively correlated with real-fault detection, while about a quarter of real faults are not coupled to common mutants [1, §3.1–3.3]. CodeT uses model-generated tests to rank model-generated programs [5, §2.1–2.2]; here tests are the product and are scored against faults the tester never sees.

Mutation-guided generation. MuTAP returns surviving mutants of the same program to the prompt so that the next round of tests kills them [6, §3.4]. Meta's ACH asks a model to write faults of a stated concern and then tests that catch them, and deploys the tests [7, §1; §2]. ACH is the closest precedent for our imagined-fault condition; both MuTAP and ACH work within one program, whereas we ask whether experience transfers to unseen programs.

Learning from experience without changing weights. Reflexion turns feedback into written reflections reused on later trials of the same task [8, §1; §4.3]. ExpeL gathers insights and successful trajectories on training tasks and evaluates on tasks not used for gathering; it finds distilled insights and retrieved examples complementary, with their relative value depending on the task [9, §5.1–5.2]. Dynamic Cheatsheet keeps a curated memory at test time; appending the whole raw history did worse than curation, and generic structured instructions alone already helped [10, §2.3; §4.3]. ACE evolves such contexts as itemised playbooks and reports that rewriting a whole context can collapse it [11, §2.2; §4.3]. None of these evaluates software testing. Our record and playbook conditions apply the idea to the choice of test inputs, and the generic-advice condition separates experience from instruction.

Testing policies and self-improvement. ILF trains a testing policy from symbolic-execution traces [12, §§4–5]; Zenith does not train model weights. CoverUp guides a fixed LLM with source, coverage and execution feedback to generate regression tests [13, §3]; coverage is not fault detection. Voyager retains executable skills within a fixed acquisition loop [14, §2]. STOP recursively rewrites its code-improvement scaffold [15, §4]. Hyperagents edits task and meta-agent code, then evaluates a fixed acquired meta-agent in another domain. Its reported cross-domain compounding comparison does not establish a statistically significant advantage [16, §§5.2–5.3].

7. Limits and next steps. Limits. All programs are short Python functions from one benchmark. The natural faults come from three code models released in 2022–2023, and the mechanical faults share operators across splits. One tester model is studied. Problems differ in difficulty and in their number of fault classes; we weight problems equally. EvalPlus inputs define the ground truth, so a fault that no augmented input exposes is invisible here. A positive result shows transfer of testing experience to unseen problems within this benchmark and budget; it does not show transfer to large systems, other languages or other models.

Improving the improver. Playbook v2 is a first, single step of an improvement procedure revising itself from its own misses. On held-out problems v2 exposed 72.7% against v1's 68.4%, but neither differs measurably from the tester shown nothing. A proper test freezes each improver, applies it to untouched starting solvers on fresh problem families, selects only on development evidence and counts all acquisition and failed proposals. Reinvesting an improved improver should be tested only after one has been shown useful.

Mathematics. The Plectis mathematical corpus contains Lean proofs and open obligations, and Plectis Atlas presents OpenAI's mathematics release with attributed outside assessments. Neither is connected to Zenith. A comparable study would need records of what was tried at a hard step, its outcome and its cost; finished proofs omit the alternatives, and a failed search does not establish falsehood.

Reproduction and collaboration. The protocol, frozen inputs, every prompt, reply, cost and execution are in the repository; scoring replays with no network calls. Zenith is an exploratory project developed with AI coding agents, and this account reflects my current understanding. I

welcome corrections and collaboration through the project's issues.

References

[1] R. Just et al. Are Mutants a Valid Substitute for Real Faults in Software Testing?. FSE, 2014.
[2] J. Liu, C. S. Xia, Y. Wang and L. Zhang. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. NeurIPS 36, 2023.
[3] M. L. Littman, R. S. Sutton and S. Singh. Predictive Representations of State. NeurIPS 14, 2001.
[4] J. Worrell. Exactly Learning Weighted Automata over a Field. Oxford teaching notes, undated.
[5] B. Chen et al. CodeT: Code Generation with Generated Tests. ICLR, 2023.
[6] A. M. Dakhel et al. Effective Test Generation Using Pre-trained Large Language Models and Mutation Testing. arXiv 2308.16557, 2023.
[7] C. Foster et al. Mutation-Guided LLM-based Test Generation at Meta. FSE Companion, 2025.
[8] N. Shinn et al. Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS 36, 2023.
[9] A. Zhao et al. ExpeL: LLM Agents Are Experiential Learners. AAAI, 2024.
[10] M. Suzgun et al. Dynamic Cheatsheet: Test-Time Learning with Adaptive Memory. arXiv 2504.07952, 2025.
[11] Q. Zhang et al. Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models. arXiv 2510.04618, 2025.
[12] J. He et al. Learning to Fuzz from Symbolic Execution with Application to Smart Contracts. CCS, 2019.
[13] J. Altmayer Pizzorno and E. D. Berger. CoverUp: Effective High Coverage Test Generation for Python. PACMSE 2 (FSE), 2025.
[14] G. Wang et al. Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv, 2023.
[15] E. Zelikman et al. Self-Taught Optimizer (STOP): Recursively Self-Improving Code Generation. COLM, 2024.
[16] J. Zhang et al. Hyperagents. arXiv, 2026.